Erlind Sejdiu
106357

# SYCL

# Portable heterogene Programmierung mit C++

## Problemstellung, Anwendungsfälle und Programmiermodell

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: [Name der betreuenden Person]
Themengruppe: [Themengruppe des Seminars]
Abgabedatum: [Datum]

# Inhaltsverzeichnis

**1 Problemstellung** **1**

**2 Forschungsfrage** **1**

**3 Stand der Technik** **2**

**4 Methode** **3**

**5 Implementierung** **4**

5.1 Stufe 1: Der erste Kernel . . . . . 4

5.2 Stufe 2: Expliziter Gerätespeicher . . . . . 4

5.3 Stufe 3: Kachelung im lokalen Speicher . . . . . 5

5.4 Stufe 4: Gerätewechsel und Kachelgröße . . . . . 6

**6 Ergebnisse** **7**

**7 Diskussion** **7**

**8 Schlussfolgerung** **8**

**Literaturverzeichnis** **9**

**A Quellcode der Fallstudie** **11**

A.1 Stufe 0: Sequentielles Ausgangsprogramm . . . . . 11

A.2 Stufe 1: USM shared und `parallel_for` . . . . . 12

A.3 Stufe 2: USM device und explizites Kopieren . . . . . 13

A.4 Stufe 3: Kachelung im lokalen Speicher . . . . . 14

A.5 Gemeinsame Hilfsfunktionen (`common.hpp`) . . . . . 16

# Abbildungsverzeichnis


1 Getrennte Codebasen je Hersteller (a) und eine SYCL-Codebasis für alle Backends (b) . . . . . 1
2 Stufe 1: Die Queue reicht einen Kernel ein, der für jedes Element von $C$ als eigenes Work-Item läuft . . . . . 4
3 Stufe 2: Pfeile zeigen, worauf ein Auftrag wartet; die Kopie von $D$ läuft parallel zu Kernel 1 . . . . . 5
4 Stufe 3: Eine Work-Group lädt die dunklen Kacheln in den lokalen Speicher 6
5 Stufe 4: Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU 6
6 Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b) . . . . . 8


# Tabellenverzeichnis


1 SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt) . . . . . 2
2 Stufen der Portierung und die dabei eingeführten Konzepte . . . . . 3
3 Leistung je Stufe in GFLOPS (Median aus fünf Läufen; Stufe 0 auf einem Kern der jeweiligen Host-CPU) . . . . . 7

# 1 Problemstellung

Rechenleistung liegt heutzutage in verschiedenen Formen vor. Neben der CPU stehen in fast jedem Rechner GPUs und andere technische Komponenten zur Verfügung, die datenparallele Aufgaben schneller bearbeiten können. Die Software, um diese Geräte jeweils anzusprechen, kann variieren, denn jeder Hersteller bringt sein eigenes Programmiermodell mit. Für die Software ergeben sich daraus drei Probleme:

1. **Herstellerbindung:** CUDA läuft nur auf NVIDIA-GPUs [1]; AMD (HIP) und Intel (Level Zero) haben eigene Schnittstellen.
2. **Mehrfache Codebasen:** Wer mehrere Hersteller unterstützen will, pflegt mehrere Implementierungen parallel (Abbildung 1a).
3. **Abstraktion gegen Kontrolle:** Offene Alternativen sind entweder sehr hardwarenah und aufwendig wie OpenCL [2] oder lassen wie OpenMP-Direktiven wenig Kontrolle über Speicher und Ausführung.

# 2 Forschungsfrage

SYCL ist ein offener Standard der Khronos Group, der diese Probleme lösen will. Mit einem einzigen Quellcode in Standard-C++ soll auf CPUs und GPUs verschiedener Hersteller dasselbe Programm laufen [3].

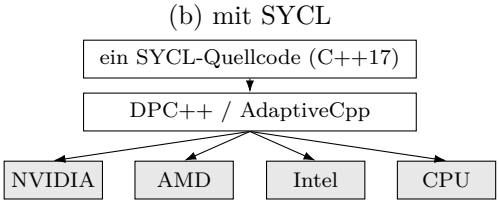



**Abbildung 1:** Getrennte Codebasen je Hersteller (a) und eine SYCL-Codebasis für alle Backends (b)

Ob ein Programm überall läuft, ist jedoch etwas anderes, als ob es überall schnell läuft. Diese Arbeit untersucht deshalb:

> Wie adressiert das Programmiermodell von SYCL die drei genannten Probleme, und erreicht ein schrittweise nach SYCL portiertes Programm neben funktionaler Portabilität auch Performance-Portabilität zwischen CPU und GPU?

Daraus ergeben sich zwei Teilfragen. (F1) Welche Konzepte des Programmiermodells werden bei einer Portierung benötigt, und welches der drei Probleme lösen sie? (F2) Welche Stufen einer Portierung lassen sich unverändert auf verschiedenen Geräten ausführen, und welche Parameter bleiben gerätespezifisch?

# 3 Stand der Technik

SYCL basiert auf C++17 [3]. Konkrete Implementierungen des Standards wie DPC++ [4] und AdaptiveCpp [5], [6] bilden den Standard über Backends wie CUDA, HIP, Level Zero, OpenCL oder OpenMP auf die Hardware ab und übersetzen den Gerätecode meist erst beim ersten Start just in time für das konkrete Gerät [7]. Den drei Problemen setzt SYCL vier Entwurfsziele entgegen. Portabilität (ein Quellcode, viele Backends), Offenheit (ein Standard mit mehreren unabhängigen Implementierungen), Produktivität (Host- und Gerätecode in einer Datei in Standard-C++) und Kontrolle (Hardwarekonzepte wie lokaler Speicher bleiben zugänglich). Tabelle 1 ordnet SYCL entlang dieser Ziele ein. Kokkos [8] und RAJA [9] verfolgen ähnliche Ziele, sind aber kein offener Industriestandard.

| | **herstellerunabh.** | **offener Standard** | **Standard-C++** | **Kontrolle** |
|---|---|---|---|---|
| CUDA | – | – | (✓) | ✓ |
| OpenCL | ✓ | ✓ | – | ✓ |
| OpenMP-Offload | ✓ | ✓ | ✓ | (✓) |
| Kokkos/RAJA | ✓ | – | ✓ | (✓) |
| **SYCL** | ✓ | ✓ | ✓ | ✓ |

**Tabelle 1:** SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt)

Eingesetzt wird SYCL vor allem im wissenschaftlichen Rechnen auf Systemen verschiedener Hersteller, etwa auf dem Supercomputer Aurora oder in GROMACS, bei der Migration von CUDA-Anwendungen mit dem Werkzeug SYCLomatic und in portablen Bibliotheken wie oneMKL [10]. Frühere Studien fanden SYCL auf vielen Plattformen konkurrenzfähig mit herstellerspezifischem Code, aber deutlich abhängig von Implementierung und Parametrierung [11], [12]. Als Maß für Performance-Portabilität hat sich die Kennzahl von Pennycook, Sewall und Lee etabliert [13]. Offen bleibt in diesen Arbeiten meist, welcher Schritt einer Portierung welchen Effekt auf welchem Gerät hat. Hier setzt die vorliegende Seminararbeit an.

# 4 Methode

Untersucht wird die Multiplikation $C = A \cdot B$ zweier $n \times n$-Matrizen in einfacher Genauigkeit [14]. Das Problem benötigt $2n^3$ Gleitkommaoperationen bei $3n^2$ Werten Datenvolumen, ist gut parallelisierbar und leicht auf Korrektheit prüfbar. Ein sequentielles C++-Programm wird in Stufen nach SYCL überführt, wobei jede Stufe nur so viele neue Konzepte einführt, wie für ihren Schritt nötig sind (Tabelle 2). Die Konzepte des Programmiermodells werden deshalb in Abschnitt 5 jeweils an der Stufe erklärt, in der sie zum ersten Mal gebraucht werden. Das beantwortet F1. Zugleich lässt sich die Wirkung jedes Schritts getrennt messen, was F2 beantwortet.

| **Stufe** | **Änderung** | **Neue Konzepte** |
|---|---|---|
| 0 | Ausgangsprogramm, drei Schleifen, ein CPU-Kern | – |
| 1 | Äußere Schleifen durch `parallel_for` ersetzt | Host und Gerät, Queue, Kernel, `range`, USM shared |
| 2 | Getrennter Gerätespeicher, explizites Kopieren | USM device, `memcpy`, Events, Abhängigkeiten, Buffer |
| 3 | Kachelung im lokalen Speicher | Command Group, `nd_range`, Work-Group, lokaler Speicher, Barriere |
| 4 | Kein neuer Code: Gerätewechsel und Variation der Kachelgröße | Geräteauswahl, Backends, Geräteabfragen, Fehlerbehandlung |

**Tabelle 2:** Stufen der Portierung und die dabei eingeführten Konzepte

Gemessen wird mit DPC++ [4] auf drei Geräten: einem AMD Ryzen 5 5600X (sechs Kerne, OpenCL-Backend), einer NVIDIA GeForce RTX 5060 Ti und einer NVIDIA L40S auf dem DACHS-Cluster der Hochschule Esslingen (beide CUDA-Backend). Jede Stufe läuft für $n \in \{256, 512, 1024, 2048\}$; nach einem getrennt erfassten ersten Lauf, der die JIT-Kompilierung enthält, folgen fünf Messläufe, von denen der Median berichtet wird. Kernel- und Transferzeit werden getrennt erfasst.

# 5 Implementierung

Ausgangspunkt (Stufe 0) sind drei verschachtelte Schleifen: Die äußeren laufen über Zeile $i$ und Spalte $j$ von $C$, die innere bildet $C_{ij} = \sum_k A_{ik}B_{kj}$. Jede weitere Stufe wird mit den Konzepten erläutert, die sie neu einführt; der Quellcode steht in Anhang A.

## 5.1 Stufe 1: Der erste Kernel

SYCL unterscheidet zwei Rollen: Der *Host* ist die CPU, auf der das Hauptprogramm läuft. Ein *Gerät* ist die Hardware, die rechnen soll, etwa eine GPU. Ein *Kernel* ist die Funktion, die auf dem Gerät vielfach parallel ausgeführt wird. Der Host übergibt Kernel über eine *Queue*, eine Warteschlange, die mit genau einem Gerät verbunden ist (Abbildung 2). Den Kernel startet `parallel_for`. Es erhält einen Indexraum (`range`), hier alle Paare $(i, j)$, und führt den Kernel für jedes Paar einmal aus. Eine solche einzelne Ausführung heißt *Work-Item*. Der Kernel ist ein gewöhnliches C++-Lambda, also eine namenlose Funktion, die direkt an der Aufrufstelle steht. Damit entfallen die beiden äußeren Schleifen, weil jedes Work-Item genau ein $C_{ij}$ berechnet; die innere Schleife wird unverändert übernommen.

Damit das Gerät auf die Matrizen zugreifen kann, werden sie mit `malloc_shared` als *Unified Shared Memory* (USM) angelegt. Ein solcher Zeiger gilt auf Host und Gerät, und die SYCL-Laufzeit, eine im Hintergrund arbeitende Bibliothek, kopiert die Daten automatisch dorthin, wo sie gebraucht werden. Die Ausführung ist asynchron: `parallel_for` kehrt sofort zurück, und der Host wartet erst mit `wait()` auf das Ergebnis. Weil Geräte keine vollständige C++-Umgebung besitzen, sind im Kernel etwa Ausnahmen und Rekursion verboten [3].

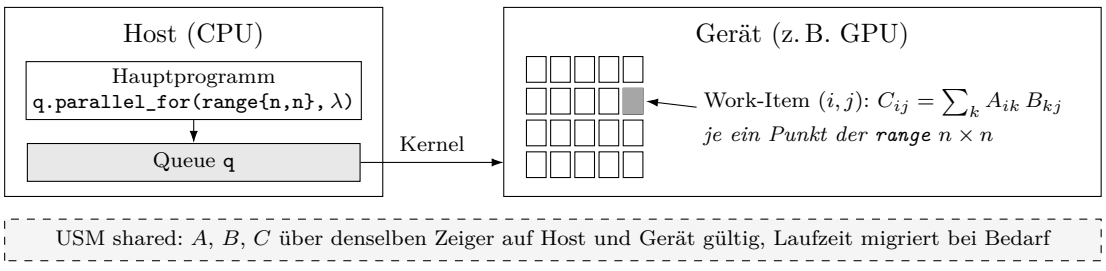



**Abbildung 2:** Stufe 1: Die Queue reicht einen Kernel ein, der für jedes Element von $C$ als eigenes Work-Item läuft

## 5.2 Stufe 2: Expliziter Gerätespeicher

Eine GPU besitzt meist eigenen, vom Host getrennten Speicher. Bei USM shared bleibt unsichtbar, wann Daten dorthin kopiert werden. Stufe 2 legt die Matrizen deshalb mit `malloc_device` nur im Gerätespeicher an und kopiert sie mit `q.memcpy` ausdrücklich

hin und zurück. So lassen sich Transfer- und Rechenzeit erstmals getrennt messen. Dadurch wird die Reihenfolge wichtig: Der Kernel darf erst starten, wenn $A$ und $B$ angekommen sind. Jeder Auftrag an die Queue liefert dafür ein *Event*, ein Objekt, das anzeigt, ob der Auftrag fertig ist. Spätere Aufträge können angeben, auf welche Events sie warten. Aus diesen Abhängigkeiten baut die Laufzeit einen Graphen, in dem jeder Knoten ein Auftrag und jede Kante eine Abhängigkeit ist (Abbildung 3). Aufträge ohne gegenseitige Abhängigkeit darf sie gleichzeitig ausführen [10].

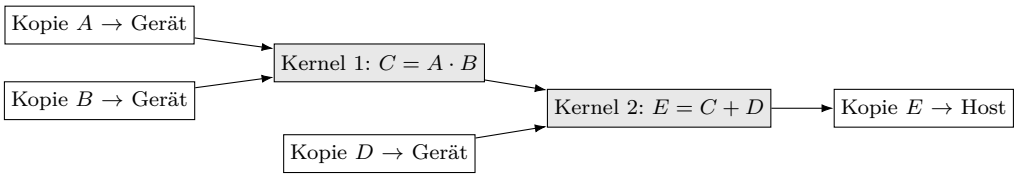



**Abbildung 3:** Stufe 2: Pfeile zeigen, worauf ein Auftrag wartet; die Kopie von $D$ läuft parallel zu Kernel 1

Bei USM gibt der Programmierer diese Abhängigkeiten selbst an. Als Alternative verwaltet ein *Buffer* die Daten, und ein Kernel greift nur über einen *Accessor* darauf zu, der angibt, ob gelesen oder geschrieben wird. Daraus leitet die Laufzeit Kopien und Abhängigkeiten selbst ab. Die Fallstudie bleibt bei USM, weil es den Zeigern des Ausgangsprogramms am nächsten ist.

## 5.3 Stufe 3: Kachelung im lokalen Speicher

In Stufe 2 liest jedes Work-Item eine ganze Zeile von $A$ und eine ganze Spalte von $B$ aus dem *globalen Speicher*, dem großen, aber langsamen Hauptspeicher des Geräts. Benachbarte Work-Items lesen dabei immer wieder dieselben Werte. Stufe 3 lässt Work-Items deshalb zusammenarbeiten und ersetzt dazu die `range` durch eine `nd_range`, die den Indexraum zusätzlich in *Work-Groups* von $T \times T$ Work-Items aufteilt. Die Work-Items einer Gruppe laufen gemeinsam auf derselben Recheneinheit und teilen sich einen kleinen, schnellen *lokalen Speicher*. Ihn zu nutzen ist, wie bei CUDA, der wichtigste Hebel für Leistung [14]. Die Matrizen werden dafür in *Kacheln*, quadratische Teilblöcke der Größe $T \times T$, zerlegt (Abbildung 4). Jede Work-Group berechnet eine Kachel von $C$. Dazu lädt sie Schritt für Schritt je eine Kachel von $A$ und $B$ in den lokalen Speicher, jedes Work-Item einen Wert. An einer *Barriere* wartet jedes Work-Item, bis alle Werte der Gruppe geladen sind, und rechnet erst dann mit den schnellen Kopien weiter. So wird jeder Wert nur noch einmal pro Work-Group statt einmal pro Work-Item aus dem globalen Speicher gelesen.

Weil lokaler Speicher vor dem Start angefordert werden muss, wird der Kernel mit `q.submit` als *Command Group* eingereicht, einer Funktion, die vorab beschreibt, was

er braucht, hier zwei Kacheln (`local_accessor`). Statt eines einfachen Index erhält der Kernel ein `nd_item` mit seiner Position in der Matrix und in der Work-Group.

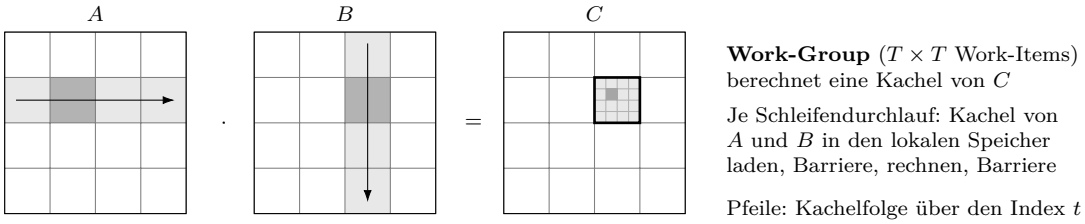



**Abbildung 4:** Stufe 3: Eine Work-Group lädt die dunklen Kacheln in den lokalen Speicher

## 5.4 Stufe 4: Gerätewechsel und Kachelgröße

Stufe 4 enthält keinen neuen Code. Sie prüft, ob die Programme der Stufen 1 bis 3 unverändert auf einem anderen Gerät laufen. SYCL spricht jedes Gerät über ein *Backend* an, die Schnittstelle des jeweiligen Herstellers, etwa CUDA für NVIDIA-GPUs oder OpenCL für die CPU. Welches Gerät eine Queue nutzt, entscheidet ein *Device Selector*, der die Geräte bewertet und das beste wählt. In der Fallstudie wurde nur die Umgebungsvariable `ONEAPI_DEVICE_SELECTOR` zwischen `opencl:cpu` und `cuda:gpu` umgestellt. Abbildung 5 zeigt, wie dieselben SYCL-Begriffe auf die beiden Geräte abgebildet werden. Auf der GPU wird eine Work-Group zu einem Thread-Block, ein Work-Item zu einem Thread. Auf der CPU übernimmt ein Kern eine Work-Group und verteilt die Work-Items auf seine Vektoreinheit (SIMD, ein Befehl für mehrere Werte); im Gleichschritt laufende Work-Items bilden eine *Sub-Group*. Das Modell entspricht weitgehend CUDA [1], was die Migration erleichtert.

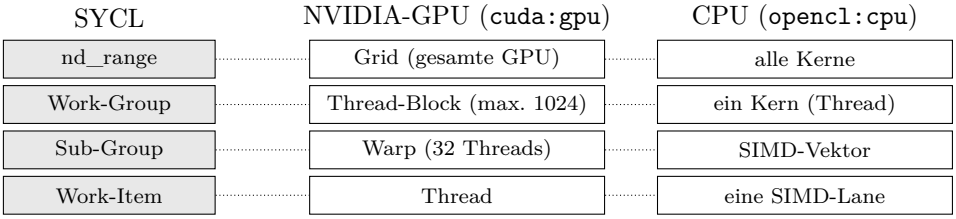



**Abbildung 5:** Stufe 4: Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU

Zusätzlich wurde Stufe 3 mit den Kachelgrößen $T \in \{8, 16, 32, 64\}$ gemessen. Bei $T = 64$ umfasst eine Work-Group $64 \cdot 64 = 4096$ Work-Items und überschreitet die Grenze von 1024 Threads je Thread-Block [15]. SYCL meldet das als C++-Ausnahme sofort beim Einreichen (*synchron*); Fehler während der Ausführung auf dem Gerät meldet es erst, wenn der Host wartet (*asynchron*). Abhilfe schafft, die maximale Work-Group-Größe zur Laufzeit abzufragen [3].

# 6 Ergebnisse

Alle Stufen liefen ohne Codeänderung und mit korrektem Ergebnis auf allen drei Geräten. Tabelle 3 zeigt die Wirkung jeder Stufe auf den beiden GPUs. Schon Stufe 1 ist bei $n = 1024$ rund 2300-mal (RTX) bzw. 10 700-mal (L40S) schneller als das sequentielle Ausgangsprogramm auf einem CPU-Kern; dieser Vergleich misst allerdings Parallelisierung und Gerätewechsel zugleich. Stufe 2 ändert die Rechenzeit nicht, macht aber den Transfer sichtbar, der bei $n = 2048$ ein Viertel (RTX) bis ein Drittel (L40S) der Rechenzeit ausmacht. Die Kachelung in Stufe 3 steigert die Leistung um 31 % auf der RTX, auf der L40S sogar um 69 %. Der erste Lauf mit JIT-Kompilierung dauert bei $n = 1024$ zwei- bis achtmal so lang wie die folgenden.

| | **RTX 5060 Ti** | | **L40S** | |
|---|---|---|---|---|
| **Stufe** | $n = 1024$ | $n = 2048$ | $n = 1024$ | $n = 2048$ |
| 0 (CPU, ein Kern) | 0,5 | – | 0,3 | – |
| 1 (USM shared) | 1105 | 1099 | 3459 | 2929 |
| 2 (USM device) | 1088 | 1083 | 3402 | 2926 |
| 3 (Kachelung, $T = 16$) | 1395 | 1418 | 4720 | 4934 |

**Tabelle 3:** Leistung je Stufe in GFLOPS (Median aus fünf Läufen; Stufe 0 auf einem Kern der jeweiligen Host-CPU)

Abbildung 6 vergleicht Transfer und Rechnung sowie alle drei Geräte. Bei kleinen Matrizen kostet der Datentransfer zur GPU mehr als die Rechnung; erst ab $n = 1024$ überwiegt die Rechenzeit (a). Die beste Kachelgröße ist geräteabhängig: Auf beiden GPUs ist $T = 16$ am schnellsten, auf der CPU $T = 32$. Bei $T = 64$ startet der Kernel auf den GPUs nicht, sodass die Performance-Portabilität auf null fällt (b). Diese ist nach Pennycook et al. [13] der harmonische Mittelwert der Effizienzen $e_i$ über die Gerätemenge $H$ und null, sobald ein Gerät nicht unterstützt wird:

$$\mathrm{PP}(a, p, H) = \frac{|H|}{\sum_{i \in H} 1/e_i(a, p)}.$$

Als Effizienz dient hier die Leistung relativ zur besten Kachelgröße auf demselben Gerät. Der höchste Wert, PP = 0,99, ergibt sich für $T = 32$.

# 7 Diskussion

Zu F1: Die Stufen zeigen, welches Konzept welches Problem löst. Die Herstellerbindung lösen Backends und Geräteauswahl (Stufe 4), die mehrfachen Codebasen Single-Source in Standard-C++ mit Kerneln als Lambdas (Stufe 1). Den Zielkonflikt zwischen Abstraktion und Kontrolle entschärft SYCL durch einen gestuften Einstieg: Eine einfache

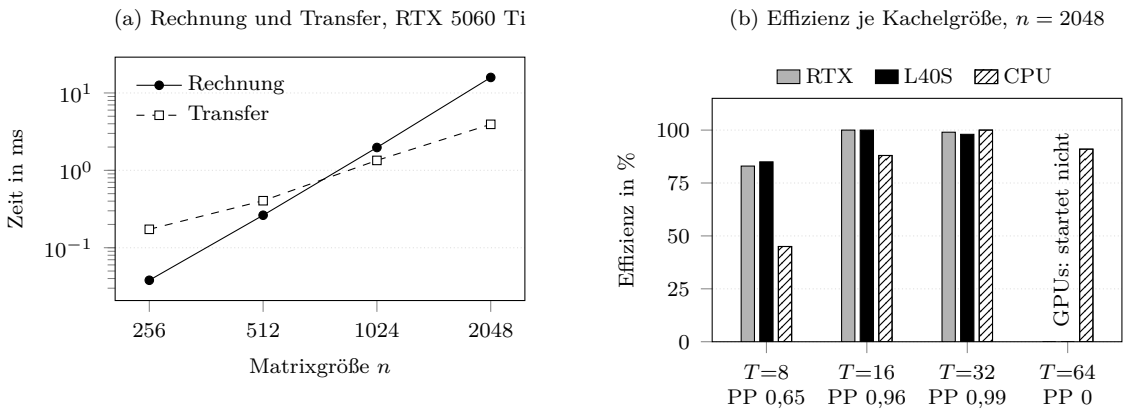



**Abbildung 6:** Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b)

`range` mit USM shared genügt für ein lauffähiges Programm, während `nd_range`, lokaler Speicher und expliziter Transfer die Kontrolle für die Optimierung zurückgeben (Stufen 2 und 3).

Zu F2: Die Fallstudie bestätigt funktionale Portabilität für alle Stufen. Performance-Portabilität ergibt sich dagegen nicht von selbst. SYCL abstrahiert die Schnittstelle, nicht die Hardware: Parameter wie die Work-Group-Größe bleiben gerätespezifisch, und ein für die CPU günstiger Wert kann auf der GPU sogar zum Abbruch führen. Dies deckt sich mit der in früheren Studien beobachteten Abhängigkeit von Implementierung und Parametrierung [11]. Selbst zwischen zwei NVIDIA-GPUs unterscheidet sich der Nutzen der Kachelung deutlich (31 % gegenüber 69 %), auch wenn die beste Kachelgröße übereinstimmt. Den größten Gewinn bringt bereits der erste, unveränderte Kernel; die hardwarenahe Kachelung lohnt sich zusätzlich, bindet das Programm aber an gerätespezifische Parameter.

Die Aussagekraft der Fallstudie ist begrenzt: Untersucht wurden ein einzelner Kernel, drei Geräte mit GPUs nur eines Herstellers und eine Implementierung. Die PP-Werte gelten nur für diese Gerätemenge [16], und ein Vergleich mit nativem CUDA-Code oder optimierten Bibliotheken fehlt.

# 8 Schlussfolgerung

SYCL erfüllt sein Versprechen „einmal schreiben, überall ausführen“ für die Korrektheit: Ein schrittweise portiertes Programm lief in allen Stufen unverändert auf CPU und GPU. Die gerätespezifische Abstimmung für hohe Leistung nimmt SYCL dem Programmierer jedoch nicht ab. Folgearbeiten sollten GPUs weiterer Hersteller, etwa die AMD MI300A des DACHS-Clusters, und eine zweite Implementierung wie AdaptiveCpp einbeziehen.

# Literaturverzeichnis


[1] J. Nickolls, I. Buck, M. Garland und K. Skadron, „Scalable Parallel Programming with CUDA," *ACM Queue*, Jg. 6, Nr. 2, S. 40–53, 2008. DOI: 10.1145/1365490.1365500.

[2] J. E. Stone, D. Gohara und G. Shi, „OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, Jg. 12, Nr. 3, S. 66–73, 2010. DOI: 10.1109/MCSE.2010.69.

[3] The Khronos SYCL Working Group. „SYCL 2020 Specification," besucht am 30. Okt. 2026. Adresse: https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html.

[4] Intel Corporation. „Intel LLVM-based Compiler (DPC++) – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/intel/llvm.

[5] A. Alpay und V. Heuveline, „SYCL beyond OpenCL: The Architecture, Current State and Future Direction of hipSYCL," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388658.

[6] AdaptiveCpp Contributors. „AdaptiveCpp – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/AdaptiveCpp/AdaptiveCpp.

[7] A. Alpay und V. Heuveline, „One Pass to Bind Them: The First Single-Pass SYCL Compiler with Unified Code Representation Across Backends," in *Proceedings of the International Workshop on OpenCL (IWOCL '23)*, 2023.

[8] H. C. Edwards, C. R. Trott und D. Sunderland, „Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, Jg. 74, Nr. 12, S. 3202–3216, 2014. DOI: 10.1016/j.jpdc.2014.07.003.

[9] D. A. Beckingsale u. a., „RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, S. 71–81.

[10] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook und X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2. Aufl. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9691-2. DOI: 10.1007/978-1-4842-9691-2.

[11] T. Deakin und S. McIntosh-Smith, „Evaluating the Performance of HPC-Style SYCL Applications," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388643.

[12] M. Breyer, A. Van Craen und D. Pflüger, „A Comparison of SYCL, OpenCL, CUDA, and OpenMP for Massively Parallel Support Vector Machine Classification on Multi-Vendor Hardware," in *Proceedings of the International Workshop on OpenCL (IWOCL '22)*, 2022.

[13] S. J. Pennycook, J. D. Sewall und V. W. Lee, „A Metric for Performance Portability," in *Proceedings of the 7th International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2016. arXiv: 1611.07409.

[14] W.-m. W. Hwu, D. B. Kirk und I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4. Aufl. Cambridge, MA: Morgan Kaufmann, 2022.

[15] NVIDIA Corporation. „CUDA C++ Programming Guide," besucht am 30. Okt. 2026. Adresse: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

[16] S. J. Pennycook, J. D. Sewall und V. W. Lee, „Implications of a Metric for Performance Portability," *Future Generation Computer Systems*, Jg. 92, S. 947–958, 2019.

# A Quellcode der Fallstudie

Die folgenden Listings zeigen den vollständigen Quellcode der Stufen 0 bis 3. Stufe 4 verwendet dieselben Programme; die Kachelgröße von Stufe 3 wird beim Übersetzen mit `-DTILE=8|16|32|64` gesetzt, das Gerät über `ONEAPI_DEVICE_SELECTOR` gewählt.

## A.1 Stufe 0: Sequentielles Ausgangsprogramm

```
// Stufe 0 - Das "existierende Programm"
//
// Klassische Matrixmultiplikation C = A * B in reinem C++, ein Kern,
// keine Parallelisierung. Dient als Ausgangspunkt und Referenz für
// Laufzeit und Korrektheit.
//
// Bild für die Folie: drei verschachtelte Schleifen. Jedes Element C[i][j]
// ist das Skalarprodukt aus Zeile i von A und Spalte j von B.

#include "common.hpp"

static void matmul(const float* A, const float* B, float* C, std::size_t n) {
  for (std::size_t i = 0; i < n; ++i)          // jede Zeile von C
    for (std::size_t j = 0; j < n; ++j) {      // jede Spalte von C
      float sum = 0.0f;
      for (std::size_t k = 0; k < n; ++k)      // Skalarprodukt
        sum += A[i * n + k] * B[k * n + j];
      C[i * n + j] = sum;
    }
}

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  std::vector<float> A(n * n), B(n * n), C(n * n);
  bench::fill_random(A.data(), n * n, 1);
  bench::fill_random(B.data(), n * n, 2);

  // Erster Lauf (zum Vergleich mit dem "ersten Lauf" der SYCL-Stufen)
  bench::Timer t0;
  matmul(A.data(), B.data(), C.data(), n);
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    matmul(A.data(), B.data(), C.data(), n);
    times.push_back(t.ms());
  }

  const bool ok = bench::verify(A.data(), B.data(), C.data(), n);
  bench::report("0_sequentiell", "CPU (1 Kern)", n, first, bench::median(times), 0.0, ok);
  return ok ? 0 : 1;
}
```

## A.2 Stufe 1: USM shared und `parallel_for`

```
// Stufe 1 - Der erste SYCL-Kernel
//
// Kleinstmögliche Änderung gegenüber Stufe 0:
//   * eine Queue anlegen (sie wählt das Gerät, z. B. die GPU)
//   * Speicher mit malloc_shared anlegen (USM "shared"): derselbe Zeiger
//     ist auf Host und Gerät gültig, die Laufzeit verschiebt die Daten
//     automatisch dorthin, wo sie gebraucht werden
//   * die beiden äußeren Schleifen (i, j) durch parallel_for ersetzen -
//     jedes C[i][j] wird von einem eigenen Work-Item berechnet
//
// Die innere Schleife (k) bleibt unverändert im Kernel.
//
// Neue Konzepte: queue, device, kernel, parallel_for, range/id, USM shared

#include <sycl/sycl.hpp>
#include "common.hpp"

int main(int argc, char** argv) {
  const auto cfg = bench::parse_args(argc, argv);
  const std::size_t n = cfg.n;

  // 1) Queue: Verbindung zu einem Gerät. Ohne Angabe nimmt SYCL das
  //    "beste" verfügbare Gerät (meist die GPU).
  sycl::queue q;
  const std::string dev = q.get_device().get_info<sycl::info::device::name>();

  // 2) Gemeinsamer Speicher (Unified Shared Memory, Variante "shared")
  float* A = sycl::malloc_shared<float>(n * n, q);
  float* B = sycl::malloc_shared<float>(n * n, q);
  float* C = sycl::malloc_shared<float>(n * n, q);
  bench::fill_random(A, n * n, 1);
  bench::fill_random(B, n * n, 2);

  // 3) Kernel: ersetzt die i- und j-Schleife aus Stufe 0
  auto run = [&] {
    q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
       const std::size_t i = idx[0];
       const std::size_t j = idx[1];
       float sum = 0.0f;
       for (std::size_t k = 0; k < n; ++k)
         sum += A[i * n + k] * B[k * n + j];
       C[i * n + j] = sum;
     }).wait();  // auf das Ende des Kernels warten
  };

  // Erster Lauf: enthält ggf. JIT-Kompilierung des Kernels und das erste
  // Verschieben der Daten auf das Gerät -> separat ausweisen!
  bench::Timer t0;
  run();
  const double first = t0.ms();

  std::vector<double> times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer t;
    run();
    times.push_back(t.ms());
  }
```

```
58
59    // Zugriff auf C vom Host aus: mit malloc_shared einfach möglich
60    const bool ok = bench::verify(A, B, C, n);
61    bench::report("1_usm_shared", dev, n, first, bench::median(times), 0.0, ok);
62
63    sycl::free(A, q);
64    sycl::free(B, q);
65    sycl::free(C, q);
66    return ok ? 0 : 1;
67  }
```

## A.3 Stufe 2: USM device und explizites Kopieren

```
1   // Stufe 2 - Speicher selbst verwalten
2   //
3   // Unterschied zu Stufe 1: Statt malloc_shared (automatisches Verschieben)
4   // wird der Speicher explizit getrennt:
5   //   * Host-Speicher (normales std::vector) für Initialisierung und Prüfung
6   //   * Geräte-Speicher (malloc_device), nur auf dem Gerät gültig
7   //   * Kopieren mit q.memcpy, Host -> Gerät und Gerät -> Host
8   //
9   // Der Kernel selbst ist identisch zu Stufe 1.
10  //
11  // Ziel für die Präsentation: Datentransfer wird sichtbar und messbar.
12  // Gerade bei kleinen Matrizen kostet das Kopieren oft mehr als die Rechnung.
13  //
14  // Neue Konzepte: malloc_device, memcpy, Trennung Host-/Geräte-Speicher
15
16  #include <sycl/sycl.hpp>
17  #include "common.hpp"
18
19  int main(int argc, char** argv) {
20    const auto cfg = bench::parse_args(argc, argv);
21    const std::size_t n = cfg.n;
22    const std::size_t bytes = n * n * sizeof(float);
23
24    sycl::queue q;
25    const std::string dev = q.get_device().get_info<sycl::info::device::name>();
26
27    // Host-Seite
28    std::vector<float> hA(n * n), hB(n * n), hC(n * n);
29    bench::fill_random(hA.data(), n * n, 1);
30    bench::fill_random(hB.data(), n * n, 2);
31
32    // Geräte-Seite
33    float* A = sycl::malloc_device<float>(n * n, q);
34    float* B = sycl::malloc_device<float>(n * n, q);
35    float* C = sycl::malloc_device<float>(n * n, q);
36
37    auto copy_in = [&] {
38      q.memcpy(A, hA.data(), bytes);
39      q.memcpy(B, hB.data(), bytes);
40      q.wait();
41    };
42    auto kernel = [&] {
43      q.parallel_for(sycl::range<2>{n, n}, [=](sycl::id<2> idx) {
44         const std::size_t i = idx[0];
```

```
45          const std::size_t j = idx[1];
46          float sum = 0.0f;
47          for (std::size_t k = 0; k < n; ++k)
48            sum += A[i * n + k] * B[k * n + j];
49          C[i * n + j] = sum;
50        }).wait();
51    };
52    auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };
53
54    // Erster Lauf komplett (Kopieren + Kernel inkl. JIT + Zurückkopieren)
55    bench::Timer t0;
56    copy_in();
57    kernel();
58    copy_out();
59    const double first = t0.ms();
60
61    // Messläufe: Kernel und Transfer getrennt erfassen
62    std::vector<double> kernel_times, transfer_times;
63    for (int r = 0; r < cfg.reps; ++r) {
64      bench::Timer tin;
65      copy_in();
66      const double in_ms = tin.ms();
67
68      bench::Timer tk;
69      kernel();
70      kernel_times.push_back(tk.ms());
71
72      bench::Timer tout;
73      copy_out();
74      transfer_times.push_back(in_ms + tout.ms());
75    }
76
77    const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
78    bench::report("2_usm_device", dev, n, first, bench::median(kernel_times),
79                  bench::median(transfer_times), ok);
80
81    sycl::free(A, q);
82    sycl::free(B, q);
83    sycl::free(C, q);
84    return ok ? 0 : 1;
85  }
```

## A.4 Stufe 3: Kachelung im lokalen Speicher

```
1   // Stufe 3 - Hardwarenah optimieren: Work-Groups und Local Memory (Tiling)
2   //
3   // Problem der Stufen 1 und 2: Jedes Work-Item liest seine komplette Zeile
4   // von A und Spalte von B aus dem langsamen globalen Speicher. Benachbarte
5   // Work-Items lesen dabei immer wieder dieselben Werte.
6   //
7   // Idee: Die Matrix wird in Kacheln (Tiles) der Größe TILE x TILE zerlegt.
8   //   * Work-Items werden zu Work-Groups von TILE x TILE zusammengefasst
9   //   * Jede Work-Group lädt gemeinsam eine Kachel von A und eine von B in
10  //     den schnellen Local Memory (auf der GPU: Shared Memory des Rechenblocks)
11  //   * Barriere: warten, bis alle Werte geladen sind
12  //   * Jedes Work-Item rechnet mit den Kacheldaten weiter
13  //   * Barriere, dann die nächste Kachel laden
```

```
14 // Dadurch wird jeder Wert aus dem globalen Speicher nur noch einmal pro
15 // Work-Group statt einmal pro Work-Item gelesen.
16 //
17 // Speicherverwaltung wie Stufe 2 - geändert hat sich nur der Kernel.
18 //
19 // Neue Konzepte: nd_range, nd_item, Work-Group, local_accessor (Local
20 // Memory), group_barrier
21 //
22 // Kachelgröße beim Kompilieren ändern, z. B. -DTILE=8 oder -DTILE=32.
23 // Die beste Wahl hängt vom Gerät ab (-> Performance-Portabilität!).
24
25 #include <sycl/sycl.hpp>
26 #include "common.hpp"
27
28 #ifndef TILE
29 #define TILE 16
30 #endif
31
32 int main(int argc, char** argv) {
33   const auto cfg = bench::parse_args(argc, argv);
34   const std::size_t n = cfg.n;
35   const std::size_t bytes = n * n * sizeof(float);
36   constexpr std::size_t T = TILE;
37
38   sycl::queue q;
39   const std::string dev = q.get_device().get_info<sycl::info::device::name>();
40
41   std::vector<float> hA(n * n), hB(n * n), hC(n * n);
42   bench::fill_random(hA.data(), n * n, 1);
43   bench::fill_random(hB.data(), n * n, 2);
44
45   float* A = sycl::malloc_device<float>(n * n, q);
46   float* B = sycl::malloc_device<float>(n * n, q);
47   float* C = sycl::malloc_device<float>(n * n, q);
48
49   // Globaler Bereich auf ein Vielfaches der Kachelgröße aufrunden, damit
50   // auch n, die nicht durch TILE teilbar sind, funktionieren.
51   const std::size_t np = (n + T - 1) / T * T;
52
53   auto copy_in = [&] {
54     q.memcpy(A, hA.data(), bytes);
55     q.memcpy(B, hB.data(), bytes);
56     q.wait();
57   };
58
59   auto kernel = [&] {
60     q.submit([&](sycl::handler& h) {
61        // Local Memory: eine Kachel von A und eine von B pro Work-Group
62        sycl::local_accessor<float, 2> tileA({T, T}, h);
63        sycl::local_accessor<float, 2> tileB({T, T}, h);
64
65        h.parallel_for(
66            sycl::nd_range<2>{{np, np}, {T, T}},  // global, lokal (Work-Group)
67            [=](sycl::nd_item<2> it) {
68              const std::size_t i = it.get_global_id(0);   // Zeile in C
69              const std::size_t j = it.get_global_id(1);   // Spalte in C
70              const std::size_t li = it.get_local_id(0);   // Position in
71              const std::size_t lj = it.get_local_id(1);   // der Kachel
72
```

```
            float sum = 0.0f;
            for (std::size_t t = 0; t < n; t += T) {
              // gemeinsam eine Kachel laden (außerhalb der Matrix: 0)
              tileA[li][lj] = (i < n && t + lj < n) ? A[i * n + t + lj] : 0.0f;
              tileB[li][lj] = (t + li < n && j < n) ? B[(t + li) * n + j] : 0.0f;
              sycl::group_barrier(it.get_group());

              // mit den Daten aus dem schnellen Local Memory rechnen
              for (std::size_t k = 0; k < T; ++k)
                sum += tileA[li][k] * tileB[k][lj];
              sycl::group_barrier(it.get_group());
            }
            if (i < n && j < n) C[i * n + j] = sum;
          });
    }).wait();
  };

  auto copy_out = [&] { q.memcpy(hC.data(), C, bytes).wait(); };

  bench::Timer t0;
  copy_in();
  kernel();
  copy_out();
  const double first = t0.ms();

  std::vector<double> kernel_times, transfer_times;
  for (int r = 0; r < cfg.reps; ++r) {
    bench::Timer tin;
    copy_in();
    const double in_ms = tin.ms();

    bench::Timer tk;
    kernel();
    kernel_times.push_back(tk.ms());

    bench::Timer tout;
    copy_out();
    transfer_times.push_back(in_ms + tout.ms());
  }

  const bool ok = bench::verify(hA.data(), hB.data(), hC.data(), n);
  bench::report("3_tiling_T" + std::to_string(T), dev, n, first,
                bench::median(kernel_times), bench::median(transfer_times), ok);

  sycl::free(A, q);
  sycl::free(B, q);
  sycl::free(C, q);
  return ok ? 0 : 1;
}
```

## A.5 Gemeinsame Hilfsfunktionen (`common.hpp`)

```
// common.hpp - gemeinsame Hilfsfunktionen für alle Stufen
//
// Enthält: Kommandozeile lesen, Matrizen befüllen, Zeitmessung,
// Ergebnisprüfung und eine einheitliche CSV-Ausgabe, damit die
// Ergebnisse aller Stufen direkt vergleichbar sind.
```

```cpp
#pragma once

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <random>
#include <string>
#include <vector>

namespace bench {

// Einstellungen aus der Kommandozeile:  ./stufeX <n> [wiederholungen]
struct Config {
  std::size_t n = 1024;   // Matrixgröße n x n
  int reps = 5;           // Messwiederholungen (nach dem Aufwärmlauf)
};

inline Config parse_args(int argc, char** argv) {
  Config c;
  if (argc > 1) c.n = std::strtoul(argv[1], nullptr, 10);
  if (argc > 2) c.reps = std::atoi(argv[2]);
  if (c.n == 0 || c.reps < 1) {
    std::fprintf(stderr, "Aufruf: %s <n> [wiederholungen]\n", argv[0]);
    std::exit(1);
  }
  return c;
}

// Matrizen mit reproduzierbaren Zufallswerten im Bereich [-1, 1] füllen.
inline void fill_random(float* m, std::size_t count, unsigned seed) {
  std::mt19937 gen(seed);
  std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
  for (std::size_t i = 0; i < count; ++i) m[i] = dist(gen);
}

// Einfache Stoppuhr in Millisekunden.
class Timer {
 public:
  Timer() : start_(clock::now()) {}
  double ms() const {
    return std::chrono::duration<double, std::milli>(clock::now() - start_).count();
  }
 private:
  using clock = std::chrono::steady_clock;
  clock::time_point start_;
};

inline double median(std::vector<double> v) {
  std::sort(v.begin(), v.end());
  const std::size_t m = v.size() / 2;
  return v.size() % 2 ? v[m] : 0.5 * (v[m - 1] + v[m]);
}

// Ergebnisprüfung: 64 zufällige Einträge von C werden in double-Genauigkeit
// nachgerechnet. Das kostet nur O(64 * n) und funktioniert auch für große n.
inline bool verify(const float* A, const float* B, const float* C, std::size_t n) {
  std::mt19937 gen(42);
```

```
65    std::uniform_int_distribution<std::size_t> idx(0, n - 1);
66    for (int s = 0; s < 64; ++s) {
67      const std::size_t i = idx(gen), j = idx(gen);
68      double ref = 0.0;
69      for (std::size_t k = 0; k < n; ++k) ref += double(A[i * n + k]) * double(B[k * n + j]);
70      const double got = C[i * n + j];
71      const double tol = 1e-3 * std::max(1.0, std::sqrt(double(n)));
72      if (std::fabs(got - ref) > tol) {
73        std::fprintf(stderr, "FEHLER bei C[%zu][%zu]: %f statt %f\n", i, j, got, ref);
74        return false;
75      }
76    }
77    return true;
78  }
79
80  // Einheitliche Ergebniszeile (CSV), damit sich alle Stufen in eine Tabelle
81  // schreiben lassen:
82  //   stufe;geraet;n;erster_lauf_ms;median_ms;transfer_ms;gflops;korrekt
83  inline void report(const std::string& stage, const std::string& device, std::size_t n,
84                     double first_ms, double median_ms, double transfer_ms, bool ok) {
85    const double gflops = 2.0 * double(n) * double(n) * double(n) / (median_ms * 1e6);
86    std::printf("%s;%s;%zu;%.3f;%.3f;%.3f;%.2f;%s\n", stage.c_str(), device.c_str(), n,
87                first_ms, median_ms, transfer_ms, gflops, ok ? "ja" : "NEIN");
88  }
89
90  } // namespace bench
```